

ARES Protocol — Full Reference

Every endpoint, every transport: REST/JSON, REST/XML, Zenoh, ROS2DDS

REST/JSON

XML

Zenoh

ROS2DDS

This is the complete reference. If you only drive a robot, the short `PROTOCOL.typ` quick-start (register / heartbeat / claim / release) is all you need. This document adds the read/query endpoints, route planning, scheduling, lease administration, and the per-transport details.

Two tiers. **Tier-1** = the four flat, key-authenticated robot calls — flat scalar bodies, a `decision` + `reason` reply. **Tier-2** = richer endpoints for graph-aware clients, with fuller JSON/XML payloads. Read-only endpoints are open; every state-changing endpoint takes a key.

Contents

1	Connection points	2
2	Shared rules	2
2.1	Resource identifiers	2
2.2	Authentication key	2
2.3	Replies and errors	2
3	Tier-1 flat protocol	2
3.1	Register — <code>POST /ares/v1/robots</code>	3
3.2	Heartbeat — <code>POST /ares/v1/robots/{id}/heartbeat</code>	3
3.3	Claim — <code>POST /ares/v1/claims/{zone node edge}</code>	3
3.4	Release — <code>POST /ares/v1/leases/release/{zone node edge}</code>	3
4	Tier-2 endpoints	3
4.1	Map and fleet (all read-only)	3
4.2	Route planning — <code>POST /routes/plan</code> (read-only)	4
4.3	Robots	4
4.4	Claims	4
4.5	Leases	4
5	Transports	5
5.1	Per-operation address map	5
5.2	REST/JSON and XML	6
5.3	Zenoh	6
5.4	ROS2DDS	6
6	Reason codes	7
7	Wire types	7
8	Operator options (opt-in)	8
8.1	State persistence	8
8.2	Admin endpoint authentication	8

1 Connection points

Transport	How to reach it
REST/JSON	<code>http://<host>:8080/ares/v1/...</code> , <code>Content-Type: application/json</code> .
REST/XML	Same URLs; send/accept <code>application/xml</code> . The REST server answers XML when the request sets the XML content-type/accept header, and there is also a standalone XML router.
Zenoh	Native queryables under the key prefix <code>ares/v1/...</code> . The server listens for Zenoh peers on <code>tcp/0.0.0.0:7447</code> .
ROS2DDS	ROS2 services <code>/ares/v1/...</code> of type <code>ares_interfaces/srv/Json</code> , bridged by <code>zenoh-bridge-ros2dds</code> .

Zenoh endpoint syntax: use `tcp/<host>:7447`, never `tcp://<host>:7447`.

2 Shared rules

2.1 Resource identifiers

Zones, nodes, and edges are UUIDs internally; client calls may use the UUID or a numeric alias. A robot id may be an integer or a UUID string. In JSON, send numeric ids/keys as quoted strings so JSON and XML agree (XML has no number type).

```
"100"           // numeric alias
"00000000-0000-0000-0000-00000000100" // UUID
```

2.2 Authentication key

A `key` authenticates the acting robot. Forms:

Key form	Meaning
integer, e.g. <code>1234</code>	Simple numeric password.
<code>did:pass=<secret></code>	Password in DID form.
<code>did:key=<...></code>	Reserved for asymmetric keys — not implemented yet (rejected).

- The key is set at registration and bound to the robot id; later calls resend it and the server checks it against that robot.
- Optional on the flat (tier-1) calls:** omit it and the server uses a shared default password — convenient but insecure. A robot that registered with a real key must keep sending it.
- Required on state-changing tier-2 calls** (assign-route, schedule, the nested claim submit) — in the request body. **Read-only** calls (health, snapshot, map lookups, list/get, route planning, claim **evaluate**) take no key.
- Admin mutation endpoints** (unregister, remove claim, release/add lease) are **open by default**; a key is required only when the operator opts in with `SYNCBOT_ADMIN_AUTH` (§8.2).

2.3 Replies and errors

Tier	Reply shape
Tier-1 (flat)	<code>decision</code> (<code>1</code> ok/grant, <code>0</code> deny) + <code>reason</code> (enum); claims add <code>blocked</code> (offending id) on denial. In XML the reply root is <code><reply></code> ; the request's outer tag is ignored on input.
Tier-2	The endpoint's data type as JSON/XML. On error: HTTP <code>400</code> with <code>{"message": "..."}.</code>

Reserved reason codes — same on every flat call: `0` = OK, `1` = mismatched key. Endpoint-specific reasons start at `2` (see §6).

3 Tier-1 flat protocol

The four robot calls. Type is in the address; body is flat scalars; reply is `decision` + `reason`. Available on every transport — XML shown; JSON/Zenoh/ROS2 carry the same fields (see §5).

3.1 Register — `POST /ares/v1/robots`

Optional `<alive>` = heartbeat interval in seconds (default 2); if no heartbeat arrives for `2×` that, the server marks the robot inactive and **auto-releases all its claims** (a background sweeper frees zones held by crashed/disconnected robots, so nothing stays stuck).

```
<reg><robot>7</robot><key>1234</key><alive>2</alive></reg>  <!-- key/alive optional; robot int or UUID -->
<reply><decision>1</decision><reason>0</reason></reply>
```

3.2 Heartbeat — `POST /ares/v1/robots/{id}/heartbeat`

Liveness + position (`zone`/`node`/`edge`); the server stamps the tick. A non-negative `zone` is a zone id; `zone = -1` means “unknown / not in any claimed zone”.

```
<hb><key>1234</key><zone>42</zone></hb>  <!-- or <zone>-1</zone> if unknown -->
<reply><decision>1</decision><reason>0</reason></reply>
```

3.3 Claim — `POST /ares/v1/claims/{zone|node|edge}`

Repeat `<id>` to claim several atomically (all-or-nothing). Optional `<access_mode>` (0 = unspecified → exclusive; 1 = exclusive default; 2 = shared; 3+ reserved → rejected) and `<lease_time>` (seconds; 0 = unlimited).

```
<claim><key>1234</key><robot>7</robot><id>42</id><id>43</id>
  <access_mode>1</access_mode><lease_time>30</lease_time></claim>
<reply><decision>0</decision><reason>2</reason><blocked>43</blocked></reply>
```

Claiming a zone reserves everything inside it, so a zone claim conflicts with a node/edge claim within that zone (and vice versa). A zone with capacity greater than 1 may be held concurrently by several robots under `access_mode=2` (shared), up to that capacity; a further shared claim past capacity is denied with reason `3` (**capacity exceeded**), while an **exclusive** claim (`access_mode=1`) on a shared zone still conflicts (reason `2`).

3.4 Release — `POST /ares/v1/leases/release/{zone|node|edge}`

```
<rel><key>1234</key><robot>7</robot><id>42</id></rel>
<reply><decision>1</decision><reason>0</reason></reply>
```

4 Tier-2 endpoints

Richer endpoints for graph-aware clients. REST paths shown; the Zenoh key and ROS2 service for each are in §5.1. **R** = read-only (open), **W** = state-changing (needs `key`).

4.1 Map and fleet (all read-only)

	Endpoint	Returns
R	<code>GET /health</code>	<code>{status, version}</code>
R	<code>GET /fleet/snapshot</code>	<code>{robots[], requests[], leases[]}</code>
R	<code>GET /zones</code> · <code>GET /zones/{id}</code>	<code>ZoneView</code> (id, numeric_id, name, kind, parent, children, nodes, properties)
R	<code>GET /nodes</code> · <code>GET /nodes/{id}</code>	<code>NodeView</code> (id, numeric_id, name, position, zones, properties)
R	<code>GET /edges</code> · <code>GET /edges/{id}</code>	<code>EdgeView</code> (id, numeric_id, source, target, directed, weight, zones, properties)

4.2 Route planning — `POST /routes/plan` (read-only)

```
request  {"start_node_id":"1001","goal_node_id":"1003","use_penalties":false}
response {"found":true,"distance":12.0,"plan":{...},"failure":null}
```

`plan` is a `RoutePlan` (traversed node/edge/zone ids + per-step costs); `failure` carries diagnostics when no route is found.

4.3 Robots

	Endpoint	Notes
W	<code>POST /robots</code>	Flat register (§3). Body <code>robot</code> + optional <code>key</code> .
R	<code>GET /robots</code> · <code>GET /robots/{id}</code>	<code>RobotState</code> / list.
W	<code>DELETE /robots/{id}</code>	Unregister. Admin path; open by default, owning-robot key via <code>?key=</code> when <code>SYNCBOT_ADMIN_AUTH</code> is set (§8.2).
W	<code>POST /robots/{id}/heartbeat</code>	Flat heartbeat (§3).
W	<code>POST /robots/{id}/route</code>	Assign a precomputed <code>RoutePlan</code> . Body <code>AssignRouteRequest</code> + <code>key</code> .
W	<code>POST /robots/{id}/schedule</code>	Schedule from a claim → <code>ScheduleDecision</code> (Proceed / Queue / Replan). Body <code>ScheduleRobotRouteRequest</code> + <code>key</code> .

```
// assign route
{"route_plan":{...},"horizon":100,"updated_at_tick":10,"key":"1234"}
// schedule
{"claim_id":10,"start_tick":20,"ticks_per_cost_unit":1.0,"access_mode":"Exclusive","key":"1234"}
```

4.4 Claims

	Endpoint	Notes
R	<code>GET /claims</code> · <code>GET /claims/{id}</code>	Active claim requests.
R	<code>POST /claims/evaluate</code>	Dry-run a nested claim; does not store it. No key.
W	<code>POST /claims</code>	Nested submit: evaluate + store if granted. Body <code>ClaimRequestWire</code> + <code>key</code> .
W	<code>POST /claims/{zone,node,edge}</code>	Flat claim (§3).
W	<code>DELETE /claims/{id}</code>	Remove a request. Admin path; open by default, owning-robot key via <code>?key=</code> when <code>SYNCBOT_ADMIN_AUTH</code> is set (§8.2).

Nested `ClaimRequestWire` (for `evaluate` / `POST /claims`):

```
{"id":10,"robot_id":1,"key":"1234","access_mode":"Exclusive","priority":10,
 "requested_at_tick":10,"window":{"start_tick":10,"end_tick":100},
 "targets":[{"kind":"Zone","resource_id":"100"}]}
```

`evaluate` returns the full `ClaimEvaluation` (`decision`, `reason`, `conflicting_*`, `blocking_target`, `diagnostics`); `POST /claims` returns the same and stores the request when `decision == Grant`.

4.5 Leases

	Endpoint	Notes
R	<code>GET /leases</code>	Active leases.
W	<code>POST /leases</code>	Add a lease directly (admin / integration).
W	<code>POST /leases/release</code>	Release by lease id — body <code>{"lease_id":501,"released_at_tick":50}</code> .

	Endpoint	Notes
W	POST /leases/release/{zone,node,edge}	Flat release by robot + resource (§3).
W	DELETE /leases/{id}	Release by lease id in the path.

5 Transports

5.1 Per-operation address map

The same operation has one address per transport. REST uses HTTP verb + path; Zenoh uses a queryable key; ROS2 uses a service name (all `ares_interfaces/srv/Json`). Where REST puts an id in the URL, Zenoh/ROS2 put it in the JSON body.

Operation	REST path	Zenoh key / ROS2 service
health	GET /health	ares/v1/health
snapshot	GET /fleet/snapshot	ares/v1/fleet/snapshot
list/get zone	GET /zones/{id}	ares/v1/zones/list/get
list/get node	GET /nodes/{id}	ares/v1/nodes/list/get
list/get edge	GET /edges/{id}	ares/v1/edges/list/get
plan route	POST /routes/plan	ares/v1/routes/plan
register	POST /robots	ares/v1/robots/register
list/get robot	GET /robots/{id}	ares/v1/robots/list/get
unregister	DELETE /robots/{id}	ares/v1/robots/unregister
heartbeat	POST /robots/{id}/heartbeat	ares/v1/robots/heartbeat
assign route	POST /robots/{id}/route	ares/v1/robots/assign_route
schedule	POST /robots/{id}/schedule	ares/v1/robots/schedule
list/get claim	GET /claims/{id}	ares/v1/claims/list/get
evaluate claim	POST /claims/evaluate	ares/v1/claims/evaluate
submit claim	POST /claims	ares/v1/claims/request
flat claim	POST /claims/{kind}	ares/v1/claims/{kind}
remove claim	DELETE /claims/{id}	ares/v1/claims/remove
list lease	GET /leases	ares/v1/leases/list
add lease	POST /leases	ares/v1/leases/add
release lease	POST /leases/release	ares/v1/leases/release
flat release	POST /leases/release/{kind}	ares/v1/leases/release/{kind}

For Zenoh/ROS2, calls that REST addresses by URL id take a small JSON object instead — `{"id":"100"}`, `{"robot_id":1}`, or `{"claim_id":10}`. The robot id on flat Zenoh/ROS2 calls is a body field (`"robot"`), since there is no URL.

5.2 REST/JSON and XML

All REST bodies and replies are JSON by default. The same routes speak XML when the request sets `Content-Type: application/xml` (and `Accept: application/xml` for GETs). XML is encoded by `quick-xml`; on input the outer element name is ignored. JSON note: the flat claim `id` is an array (`"id":[42]`, or `[42,43]`); XML repeats `<id>`.

5.3 Zenoh

Native queryables under `ares/v1/...`, JSON payloads. A query with no input sends an empty payload. Replies are JSON; errors are Zenoh error replies carrying `{"message":"..."}`.

```
query ares/v1/claims/zone payload: {"key":"1234","robot":"7","id":[42]}
reply {"decision":1,"reason":0}
```

5.4 ROS2DDS

The server stays pure Zenoh; ROS2 clients reach it through `zenoh-bridge-ros2dds`, which maps a ROS2 service request to a Zenoh query. One generic service type carries everything:

```
# ares_interfaces/srv/Json (ros/ares_interfaces/srv/Json.srv)
string request      # JSON request object as a string; "{}" for no input
---
bool  success       # false when ARES returns an error
string response     # JSON response on success; error text on failure
```

```
ros2 service call /ares/v1/robots/register ares_interfaces/srv/Json \
  "{request: '{\"robot\": \"7\", \"key\": \"1234\"}'}"
# success=true response='{"decision":1,"reason":0}'

ros2 service call /ares/v1/claims/zone ares_interfaces/srv/Json \
  "{request: '{\"key\": \"1234\", \"robot\": \"7\", \"id\": [42]}'}"
# success=true response='{"decision":1,"reason":0}'
```

Tier-2 endpoints ride the same generic service — the `request` string carries the fuller JSON body, and `response` carries the endpoint's data type as a JSON string. Service names come from the §5.1 map; ids that REST puts in the URL go in the body:

```
# read, no input — fleet snapshot (request "{}")
ros2 service call /ares/v1/fleet/snapshot ares_interfaces/srv/Json \
  "{request: '{}'}"
# success=true response='{"robots":[...],"requests":[...],"leases":[...]}'

# read by id — one zone (id-addressed calls take {"id":"..."})
ros2 service call /ares/v1/zones/get ares_interfaces/srv/Json \
  "{request: '{\"id\": \"3\"}'}"
# success=true response='{"id":"...", "numeric_id":3, "name":"...", "kind":"...", ...}'

# route planning
ros2 service call /ares/v1/routes/plan ares_interfaces/srv/Json \
  "{request: '{\"start_node_id\": \"1001\", \"goal_node_id\": \"1003\"}'}"
# success=true response='{"found":true, "distance":12.0, "plan":[...], "failure":null}'

# nested claim submit — ClaimRequestWire body, key required
ros2 service call /ares/v1/claims/request ares_interfaces/srv/Json \
  "{request: '{\"id\":10, \"robot_id\":1, \"key\": \"1234\", \"access_mode\": \"Exclusive\", \"priority\":10, \"requested_at_tick\":10, \"window\": {\"start_tick\":10, \"end_tick\":100}, \"targets\": [{\"kind\": \"Zone\", \"resource_id\": \"100\"}]}'}"
# success=true response='{"decision":"Grant", "reason":"...", "blocking_target":null, "diagnostics":[]}'

# unregister (admin) — body field is robot_id; add "key" only when SYNCBOT_ADMIN_AUTH is set
ros2 service call /ares/v1/robots/unregister ares_interfaces/srv/Json \
  "{request: '{\"robot_id\":7}'}"
# success=true response='true' (bool: whether a robot was removed)
```

Connect the bridge to the ARES host (note `tcp/`, not `tcp://`):

```
zenoh-bridge-ros2dds -e tcp/<host>:7447
```

`ros2 service list` may not show the ARES services in the pure-Zenoh setup — that is expected. Call the `/ares/v1/...` service directly; the bridge creates the route when it sees the client.

5.4.1 Troubleshooting

Symptom	Fix
Unicast not supported for tcp: protocol	Endpoint used <code>tcp://...</code> ; use <code>tcp/...</code> .
service not listed by <code>ros2 service list</code>	Expected; call the service directly.
waiting for service to become available...	Keep the bridge running, then call directly.
received invalid request ... less than 20 bytes	Use CycloneDDS: <code>export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp</code> .
no response	Check firewall/NAT; confirm the bridge can reach <code>tcp/<host>:7447</code> .

5.4.2 Response CDR encoding

ARES replies with a CDR-encoded `ares_interfaces/srv/Json_Response`:

```
00 01 00 00    CDR little-endian header
01            bool success
00 00 00      padding to 4 bytes
<u32 len incl. NUL> string length
<UTF-8 JSON bytes>
00            trailing NUL
<padding to 4 bytes>
```

6 Reason codes

`0` = OK and `1` = mismatched key on every flat call. Endpoint-specific:

Call	Reasons (≥ 2)
register	2 already registered · 3 bad id (not int/UIID) · 4 unsupported key scheme
heartbeat	2 not registered
claim	2 conflict · 3 capacity exceeded · 4 unknown resource · 5 bad request (no id / unsupported <code>access_mode</code>)
release	2 no such lease · 3 unknown / bad

Tier-2 endpoints report failure as HTTP `400` + `{"message": "..."}` rather than a reason code.

7 Wire types

Type	Fields
<code>FlatRegister</code>	<code>robot</code> , <code>key?</code> , <code>alive?</code> (heartbeat interval s, default 2)
<code>FlatHeartbeat</code>	<code>key?</code> , one of <code>zone</code> / <code>node</code> / <code>edge</code>
<code>FlatClaim</code>	<code>key?</code> , <code>robot</code> , <code>id[]</code> , <code>access_mode?</code> , <code>lease_time?</code>
<code>FlatRelease</code>	<code>key?</code> , <code>robot</code> , <code>id</code>
<code>FlatReply</code>	<code>decision</code> (1/0), <code>reason</code> (enum), <code>blocked?</code>
<code>PlanRouteRequest</code>	<code>start_node_id</code> , <code>goal_node_id</code> , <code>use_penalties</code>

Type	Fields
ClaimRequestWire	id, robot_id, access_mode, priority, window, targets[], key?
AssignRouteRequest	route_plan, horizon, updated_at_tick, key?
ScheduleRobotRouteRequest	claim_id, start_tick, ticks_per_cost_unit, access_mode, key?
ReleaseLeaseRequest	lease_id, released_at_tick?

Common enums: claim target kind `Zone / Node / Edge`; access mode `Exclusive / Shared`; lease disposition `Active / Released / Expired / Revoked`; robot progress `Idle / FollowingRoute / Waiting / Queued / Blocked / Replanning`.

8 Operator options (opt-in)

Two server-side options that an operator may enable at deploy time. **Both are off by default; the protocol seen by existing clients is unchanged.**

8.1 State persistence

By default the server is **fully ephemeral**: all state lives in memory and is lost on restart. This is the default and is unchanged.

Optionally, set the env var `SYNCBOT_STATE` to a JSON file path to persist state across restarts. When set, on boot the server **restores** registrations, auth keys, UUID→id mappings, and active claims/leases from that file; while running it **flushes every 2 seconds** (atomic write) and once more on shutdown. If `SYNCBOT_STATE` is unset, nothing is written and the server stays fully ephemeral (state lost on restart — the default).

8.2 Admin endpoint authentication

The admin mutation endpoints — unregister a robot (`DELETE /robots/{id}`), remove a claim (`DELETE /claims/{id}`), release a lease by id (`DELETE /leases/{id}` / `POST /leases/release`), and add a lease (`POST /leases`) — are **open by default**: no key is required. This is the default and is unchanged.

Optionally, an operator may set the env var `SYNCBOT_ADMIN_AUTH=1` (also accepts `true` / `yes`) to require the owning robot's key on these endpoints — supplied as `?key=...` on the body-less DELETES and `POST /leases`, or in the body where the endpoint already takes one. With auth enabled:

- a robot that registered **without** a key stays openly manageable (it is bound to the shared default key);
- a robot that registered **with** a key is protected — a missing or wrong key is rejected (reason `1` / mismatched-key error).

Both options above are **opt-in**. Leave `SYNCBOT_STATE` and `SYNCBOT_ADMIN_AUTH` unset and the server behaves exactly as before: ephemeral state and open admin endpoints.